

Week 5 Task

Q1. What are the key limitations of traditional MapReduce that make Spark a preferred choice for modern big data processing?

Ans: Spark is preferred because it processes data much faster and handles complex workloads more efficiently than traditional MapReduce.

MapReduce was designed for processing large amounts of data, but it has some limitations. One major issue is that it writes intermediate results to disk after every stage. Disk operations are much slower than memory operations, which increases execution time.

Another limitation is that iterative tasks such as machine learning algorithms require the same data to be read from disk repeatedly. This makes processing inefficient and time-consuming.

Spark solves these problems by keeping data in memory whenever possible. It also provides easy-to-use APIs for SQL queries, machine learning, streaming, and graph processing, making development much simpler.

Q2. Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Ans: Spark speeds up machine learning by storing frequently used data in memory instead of repeatedly reading it from disk.

Many machine learning algorithms perform multiple iterations on the same dataset. In a disk-based system, data must be loaded from storage during every iteration, which creates significant overhead.

Spark allows datasets to be cached in memory. Once the data is loaded, subsequent iterations can directly access the cached data without additional disk reads.

This reduces I/O operations and improves performance considerably. As a result, machine learning tasks that may take hours on disk-based systems can often be completed much faster using Spark.

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on user_id and transaction_date.

Ans: The `dropDuplicates()` function removes repeated records based on selected columns.

```
df.dropDuplicates(["user_id"]).show()
```

user_id	city	age	subscription	region	product_category	price	status	email	username	store_id
101	Delhi	22	Premium	North	Electronics	45000	Active	rahul@gmail.com	rahul	S101
102	Mumbai	28	Premium	West	Clothing	2500	Active	priya@gmail.com	priya	S102
103	Pune	35	Basic	West	Electronics	30000	NULL	amit@gmail.com	amit	S103
104	Delhi	19	Premium	North	Books	800	Active	neha@gmail.com	neha	S101
105	Jaipur	31	Basic	West	Clothing	NULL	Inactive	NULL	rohit	S104
106	Mumbai	24	Premium	West	Electronics	55000	Active	simran@gmail.com		S102
107	Chennai	27	Premium	South	Books	1200	Active	kiran@gmail.com	kiran	S105
108	Bangalore	29	Premium	South	Electronics	40000	Active	ananya@gmail.com	ananya	S106
109	Hyderabad	21	Basic	South	Clothing	1800	NULL	arjun@gmail.com	arjun	S105
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103

This checks the user_id column and removes duplicate customer records. If the same user appears multiple times with identical information, Spark keeps one record and removes the duplicate entries. Cleaning duplicates is important because repeated records can affect reports and business analysis.

Q4. Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

```
from pyspark.sql.functions import avg

df.filter(df.region == "West") \
  .groupBy("product_category") \
  .agg(avg("price").alias("average_sales")) \
  .show()
```

product_category	average_sales
Electronics	38750.0
Clothing	2500.0

Ans:

This query focuses only on customers from the West region. It groups transactions by product category and calculates the average sales amount. The result helps identify which categories perform better in a specific region.

Q5. What is the difference between .na.drop() and .na.fill()?

Ans: One removes rows with missing values, while the other replaces missing values.

na.drop() is used when rows containing null values are not useful and can be safely removed.

na.fill() is used when you want to keep the row but replace missing values with a default value.

Example:

```
df.na.fill({"status": "Unknown"}).show()
```

user_id	city	age	subscription	region	product_category	price	status	email	username	store_id
101	Delhi	22	Premium	North	Electronics	45000	Active	rahul@gmail.com	rahul	S101
102	Mumbai	28	Premium	West	Clothing	2500	Active	priya@gmail.com	priya	S102
103	Pune	35	Basic	West	Electronics	30000	Unknown	amit@gmail.com	amit	S103
104	Delhi	19	Premium	North	Books	800	Active	neha@gmail.com	neha	S101
105	Jaipur	31	Basic	West	Clothing	NULL	Inactive	NULL	rohit	S104
106	Mumbai	24	Premium	West	Electronics	55000	Active	simran@gmail.com		S102
107	Chennai	27	Premium	South	Books	1200	Active	kiran@gmail.com	kiran	S105
108	Bangalore	29	Premium	South	Electronics	40000	Active	ananya@gmail.com	ananya	S106
109	Hyderabad	21	Basic	South	Clothing	1800	Unknown	arjun@gmail.com	arjun	S105
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103

Some transactions may not have a status recorded. Instead of deleting those records, this command replaces the missing values with "Unknown". This preserves the data while making it easier to analyze.

Q6. Write a query to find the total count of records for each city where the count is greater than 100.

```
from pyspark.sql.functions import count
df.groupBy("city") \
  .agg(count("*").alias("total_records")) \
  .show()
```

city	total_records
Mumbai	2
Pune	3
Delhi	2
Jaipur	1
Bangalore	1
Chennai	1
Hyderabad	1

This groups customers according to their city and counts the number of records in each group. Businesses often use this type of analysis to understand where most customers are located.

Q7. How does the immutability of Spark DataFrames affect data cleaning steps?

Ans: Spark never changes the original DataFrame; every modification creates a new one.

When performing cleaning operations such as dropping columns or renaming them, Spark creates a new DataFrame rather than modifying the existing one.

For example:

```
new_df = df.drop("age")
```

The original DataFrame remains unchanged.

This design improves reliability, supports fault tolerance, and allows Spark to optimize execution efficiently.

Q8. Write a Spark command to filter a dataset where age is between 18 and 30 and subscription is 'Premium'.

Ans: Multiple conditions can be combined using logical operators.

```
df.filter(
  (df.age >= 18) &
  (df.age <= 30) &
  (df.subscription == "Premium")
).show()
```

user_id	city	age	subscription	region	product_category	price	status	email	username	store_id
101	Delhi	22	Premium	North	Electronics	45000	Active	rahul@gmail.com	rahul	S101
102	Mumbai	28	Premium	West	Clothing	2500	Active	priya@gmail.com	priya	S102
104	Delhi	19	Premium	North	Books	800	Active	neha@gmail.com	neha	S101
106	Mumbai	24	Premium	West	Electronics	55000	Active	simran@gmail.com	simran	S102
107	Chennai	27	Premium	South	Books	1200	Active	kiran@gmail.com	kiran	S105
108	Bangalore	29	Premium	South	Electronics	40000	Active	ananya@gmail.com	ananya	S106
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103

This query returns customers who are between 18 and 30 years old and have a Premium subscription. Filtering data like this helps target specific customer segments.

Q9. Why is it often better to handle null values before performing aggregations like sum() or avg()?

Ans: Cleaning null values first helps produce more reliable results.

Null values can affect calculations and may lead to incomplete or misleading outputs. For example, missing prices can distort averages or revenue calculations.

A common approach is to replace null numeric values with 0 or another meaningful value before aggregation.

```
df = df.na.fill({"price": 0})
```

Doing this ensures that calculations are based on consistent data.

Q10. Write the code to cast raw_timestamp to TimestampType and rename it to event_time.

Ans: The column is converted to the correct data type and given a more meaningful name.

```
from pyspark.sql.types import TimestampType
from pyspark.sql.functions import col

df3 = (
    df2.withColumn(
        "raw_timestamp",
        col("raw_timestamp").cast(TimestampType())
    )
    .withColumnRenamed("raw_timestamp", "event_time")
)

df3.show()
```

user_id	city	age	subscription	region	product_category	price	status	email	username	store_id	event_time
101	Delhi	22	Premium	North	Electronics	45000	Active	rahul@gmail.com	rahul	S101	2026-06-06 19:19:...
102	Mumbai	28	Premium	West	Clothing	2500	Active	priya@gmail.com	priya	S102	2026-06-06 19:19:...
103	Pune	35	Basic	West	Electronics	30000	NULL	amit@gmail.com	amit	S103	2026-06-06 19:19:...
104	Delhi	19	Premium	North	Books	800	Active	neha@gmail.com	neha	S101	2026-06-06 19:19:...
105	Jaipur	31	Basic	West	Clothing	NULL	Inactive	NULL	rohit	S104	2026-06-06 19:19:...
106	Mumbai	24	Premium	West	Electronics	55000	Active	slrkan@gmail.com		S102	2026-06-06 19:19:...
107	Chennai	27	Premium	South	Books	1200	Active	kiran@gmail.com	kiran	S105	2026-06-06 19:19:...
108	Bangalore	29	Premium	South	Electronics	40000	Active	ananya@gmail.com	ananya	S106	2026-06-06 19:19:...
109	Hyderabad	21	Basic	South	Clothing	1800	NULL	arjun@gmail.com	arjun	S105	2026-06-06 19:19:...
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103	2026-06-06 19:19:...
110	Pune	26	Premium	West	Electronics	35000	Active	meera@gmail.com	meera	S103	2026-06-06 19:19:...

This converts the timestamp into Spark's Timestamp format and renames the column to event_time.

Q11. Explain the Shuffle process during a grouping operation. Why is it considered a wide transformation?

Ans: Shuffle moves data across partitions so related records can be processed together.

When a groupBy() operation is executed, records with the same key may exist in different partitions. Spark must move these records to the same location before grouping them.

This movement of data between executors is called a shuffle.

It is considered a wide transformation because data from multiple partitions is required to produce the final result. Since network communication is involved, shuffling is often one of the most expensive operations in Spark.

Q12. Write a code snippet that removes rows where email is null OR username is empty.

Ans: Filter out incomplete records to improve data quality.

```
1) from pyspark.sql.functions import col
2)
3)
4)
5)
6)
7)
8)
9)
10)
11)
12)
13)
14)
15)
16)
17)
18)
19)
20)
21)
22)
23)
24)
25)
26)
27)
28)
29)
30)
31)
32)
33)
34)
35)
36)
37)
38)
39)
40)
41)
42)
43)
44)
45)
46)
47)
48)
49)
50)
51)
52)
53)
54)
55)
56)
57)
58)
59)
60)
61)
62)
63)
64)
65)
66)
67)
68)
69)
70)
71)
72)
73)
74)
75)
76)
77)
78)
79)
80)
81)
82)
83)
84)
85)
86)
87)
88)
89)
90)
91)
92)
93)
94)
95)
96)
97)
98)
99)
100)
101)
102)
103)
104)
105)
106)
107)
108)
109)
110)
111)
112)
113)
114)
115)
116)
117)
118)
119)
120)
121)
122)
123)
124)
125)
126)
127)
128)
129)
130)
131)
132)
133)
134)
135)
136)
137)
138)
139)
140)
141)
142)
143)
144)
145)
146)
147)
148)
149)
150)
151)
152)
153)
154)
155)
156)
157)
158)
159)
160)
161)
162)
163)
164)
165)
166)
167)
168)
169)
170)
171)
172)
173)
174)
175)
176)
177)
178)
179)
180)
181)
182)
183)
184)
185)
186)
187)
188)
189)
190)
191)
192)
193)
194)
195)
196)
197)
198)
199)
200)
201)
202)
203)
204)
205)
206)
207)
208)
209)
210)
211)
212)
213)
214)
215)
216)
217)
218)
219)
220)
221)
222)
223)
224)
225)
226)
227)
228)
229)
230)
231)
232)
233)
234)
235)
236)
237)
238)
239)
240)
241)
242)
243)
244)
245)
246)
247)
248)
249)
250)
251)
252)
253)
254)
255)
256)
257)
258)
259)
260)
261)
262)
263)
264)
265)
266)
267)
268)
269)
270)
271)
272)
273)
274)
275)
276)
277)
278)
279)
280)
281)
282)
283)
284)
285)
286)
287)
288)
289)
290)
291)
292)
293)
294)
295)
296)
297)
298)
299)
300)
301)
302)
303)
304)
305)
306)
307)
308)
309)
310)
311)
312)
313)
314)
315)
316)
317)
318)
319)
320)
321)
322)
323)
324)
325)
326)
327)
328)
329)
330)
331)
332)
333)
334)
335)
336)
337)
338)
339)
340)
341)
342)
343)
344)
345)
346)
347)
348)
349)
350)
351)
352)
353)
354)
355)
356)
357)
358)
359)
360)
361)
362)
363)
364)
365)
366)
367)
368)
369)
370)
371)
372)
373)
374)
375)
376)
377)
378)
379)
380)
381)
382)
383)
384)
385)
386)
387)
388)
389)
390)
391)
392)
393)
394)
395)
396)
397)
398)
399)
400)
401)
402)
403)
404)
405)
406)
407)
408)
409)
410)
411)
412)
413)
414)
415)
416)
417)
418)
419)
420)
421)
422)
423)
424)
425)
426)
427)
428)
429)
430)
431)
432)
433)
434)
435)
436)
437)
438)
439)
440)
441)
442)
443)
444)
445)
446)
447)
448)
449)
450)
451)
452)
453)
454)
455)
456)
457)
458)
459)
460)
461)
462)
463)
464)
465)
466)
467)
468)
469)
470)
471)
472)
473)
474)
475)
476)
477)
478)
479)
480)
481)
482)
483)
484)
485)
486)
487)
488)
489)
490)
491)
492)
493)
494)
495)
496)
497)
498)
499)
500)
501)
502)
503)
504)
505)
506)
507)
508)
509)
510)
511)
512)
513)
514)
515)
516)
517)
518)
519)
520)
521)
522)
523)
524)
525)
526)
527)
528)
529)
530)
531)
532)
533)
534)
535)
536)
537)
538)
539)
540)
541)
542)
543)
544)
545)
546)
547)
548)
549)
550)
551)
552)
553)
554)
555)
556)
557)
558)
559)
560)
561)
562)
563)
564)
565)
566)
567)
568)
569)
570)
571)
572)
573)
574)
575)
576)
577)
578)
579)
580)
581)
582)
583)
584)
585)
586)
587)
588)
589)
590)
591)
592)
593)
594)
595)
596)
597)
598)
599)
600)
601)
602)
603)
604)
605)
606)
607)
608)
609)
610)
611)
612)
613)
614)
615)
616)
617)
618)
619)
620)
621)
622)
623)
624)
625)
626)
627)
628)
629)
630)
631)
632)
633)
634)
635)
636)
637)
638)
639)
640)
641)
642)
643)
644)
645)
646)
647)
648)
649)
650)
651)
652)
653)
654)
655)
656)
657)
658)
659)
660)
661)
662)
663)
664)
665)
666)
667)
668)
669)
670)
671)
672)
673)
674)
675)
676)
677)
678)
679)
680)
681)
682)
683)
684)
685)
686)
687)
688)
689)
690)
691)
692)
693)
694)
695)
696)
697)
698)
699)
700)
701)
702)
703)
704)
705)
706)
707)
708)
709)
710)
711)
712)
713)
714)
715)
716)
717)
718)
719)
720)
721)
722)
723)
724)
725)
726)
727)
728)
729)
730)
731)
732)
733)
734)
735)
736)
737)
738)
739)
740)
741)
742)
743)
744)
745)
746)
747)
748)
749)
750)
751)
752)
753)
754)
755)
756)
757)
758)
759)
760)
761)
762)
763)
764)
765)
766)
767)
768)
769)
770)
771)
772)
773)
774)
775)
776)
777)
778)
779)
780)
781)
782)
783)
784)
785)
786)
787)
788)
789)
790)
791)
792)
793)
794)
795)
796)
797)
798)
799)
800)
801)
802)
803)
804)
805)
806)
807)
808)
809)
810)
811)
812)
813)
814)
815)
816)
817)
818)
819)
820)
821)
822)
823)
824)
825)
826)
827)
828)
829)
830)
831)
832)
833)
834)
835)
836)
837)
838)
839)
840)
841)
842)
843)
844)
845)
846)
847)
848)
849)
850)
851)
852)
853)
854)
855)
856)
857)
858)
859)
860)
861)
862)
863)
864)
865)
866)
867)
868)
869)
870)
871)
872)
873)
874)
875)
876)
877)
878)
879)
880)
881)
882)
883)
884)
885)
886)
887)
888)
889)
890)
891)
892)
893)
894)
895)
896)
897)
898)
899)
900)
901)
902)
903)
904)
905)
906)
907)
908)
909)
910)
911)
912)
913)
914)
915)
916)
917)
918)
919)
920)
921)
922)
923)
924)
925)
926)
927)
928)
929)
930)
931)
932)
933)
934)
935)
936)
937)
938)
939)
940)
941)
942)
943)
944)
945)
946)
947)
948)
949)
950)
951)
952)
953)
954)
955)
956)
957)
958)
959)
960)
961)
962)
963)
964)
965)
966)
967)
968)
969)
970)
971)
972)
973)
974)
975)
976)
977)
978)
979)
980)
981)
982)
983)
984)
985)
986)
987)
988)
989)
990)
991)
992)
993)
994)
995)
996)
997)
998)
999)
1000)
```

This removes records where the email address is missing or the username is empty. Such incomplete records can affect data quality, so cleaning them before analysis is a good practice

Q13. How do you use .agg() to calculate min, max, and mean of the price column?

Ans:

```
1) from pyspark.sql.functions import min, max, avg
2)
3)
4)
5)
6)
7)
8)
9)
10)
11)
12)
13)
14)
15)
16)
17)
18)
19)
20)
21)
22)
23)
24)
25)
26)
27)
28)
29)
30)
31)
32)
33)
34)
35)
36)
37)
38)
39)
40)
41)
42)
43)
44)
45)
46)
47)
48)
49)
50)
51)
52)
53)
54)
55)
56)
57)
58)
59)
60)
61)
62)
63)
64)
65)
66)
67)
68)
69)
70)
71)
72)
73)
74)
75)
76)
77)
78)
79)
80)
81)
82)
83)
84)
85)
86)
87)
88)
89)
90)
91)
92)
93)
94)
95)
96)
97)
98)
99)
100)
101)
102)
103)
104)
105)
106)
107)
108)
109)
110)
111)
112)
113)
114)
115)
116)
117)
118)
119)
120)
121)
122)
123)
124)
125)
126)
127)
128)
129)
130)
131)
132)
133)
134)
135)
136)
137)
138)
139)
140)
141)
142)
143)
144)
145)
146)
147)
148)
149)
150)
151)
152)
153)
154)
155)
156)
157)
158)
159)
160)
161)
162)
163)
164)
165)
166)
167)
168)
169)
170)
171)
172)
173)
174)
175)
176)
177)
178)
179)
180)
181)
182)
183)
184)
185)
186)
187)
188)
189)
190)
191)
192)
193)
194)
195)
196)
197)
198)
199)
200)
201)
202)
203)
204)
205)
206)
207)
208)
209)
210)
211)
212)
213)
214)
215)
216)
217)
218)
219)
220)
221)
222)
223)
224)
225)
226)
227)
228)
229)
230)
231)
232)
233)
234)
235)
236)
237)
238)
239)
240)
241)
242)
243)
244)
245)
246)
247)
248)
249)
250)
251)
252)
253)
254)
255)
256)
257)
258)
259)
260)
261)
262)
263)
264)
265)
266)
267)
268)
269)
270)
271)
272)
273)
274)
275)
276)
277)
278)
279)
280)
281)
282)
283)
284)
285)
286)
287)
288)
289)
290)
291)
292)
293)
294)
295)
296)
297)
298)
299)
300)
301)
302)
303)
304)
305)
306)
307)
308)
309)
310)
311)
312)
313)
314)
315)
316)
317)
318)
319)
320)
321)
322)
323)
324)
325)
326)
327)
328)
329)
330)
331)
332)
333)
334)
335)
336)
337)
338)
339)
340)
341)
342)
343)
344)
345)
346)
347)
348)
349)
350)
351)
352)
353)
354)
355)
356)
357)
358)
359)
360)
361)
362)
363)
364)
365)
366)
367)
368)
369)
370)
371)
372)
373)
374)
375)
376)
377)
378)
379)
380)
381)
382)
383)
384)
385)
386)
387)
388)
389)
390)
391)
392)
393)
394)
395)
396)
397)
398)
399)
400)
401)
402)
403)
404)
405)
406)
407)
408)
409)
410)
411)
412)
413)
414)
415)
416)
417)
418)
419)
420)
421)
422)
423)
424)
425)
426)
427)
428)
429)
430)
431)
432)
433)
434)
435)
436)
437)
438)
439)
440)
441)
442)
443)
444)
445)
446)
447)
448)
449)
450)
451)
452)
453)
454)
455)
456)
457)
458)
459)
460)
461)
462)
463)
464)
465)
466)
467)
468)
469)
470)
471)
472)
473)
474)
475)
476)
477)
478)
479)
480)
481)
482)
483)
484)
485)
486)
487)
488)
489)
490)
491)
492)
493)
494)
495)
496)
497)
498)
499)
500)
501)
502)
503)
504)
505)
506)
507)
508)
509)
510)
511)
512)
513)
514)
515)
516)
517)
518)
519)
520)
521)
522)
523)
524)
525)
526)
527)
528)
529)
530)
531)
532)
533)
534)
535)
536)
537)
538)
539)
540)
541)
542)
543)
544)
545)
546)
547)
548)
549)
550)
551)
552)
553)
554)
555)
556)
557)
558)
559)
560)
561)
562)
563)
564)
565)
566)
567)
568)
569)
570)
571)
572)
573)
574)
575)
576)
577)
578)
579)
580)
581)
582)
583)
584)
585)
586)
587)
588)
589)
590)
591)
592)
593)
594)
595)
596)
597)
598)
599)
600)
601)
602)
603)
604)
605)
606)
607)
608)
609)
610)
611)
612)
613)
614)
615)
616)
617)
618)
619)
620)
621)
622)
623)
624)
625)
626)
627)
628)
629)
630)
631)
632)
633)
634)
635)
636)
637)
638)
639)
640)
641)
642)
643)
644)
645)
646)
647)
648)
649)
650)
651)
652)
653)
654)
655)
656)
657)
658)
659)
660)
661)
662)
663)
664)
665)
666)
667)
668)
669)
670)
671)
672)
673)
674)
675)
676)
677)
678)
679)
680)
681)
682)
683)
684)
685)
686)
687)
688)
689)
690)
691)
692)
693)
694)
695)
696)
697)
698)
699)
700)
701)
702)
703)
704)
705)
706)
707)
708)
709)
710)
711)
712)
713)
714)
715)
716)
717)
718)
719)
720)
721)
722)
723)
724)
725)
726)
727)
728)
729)
730)
731)
732)
733)
734)
735)
736)
737)
738)
739)
740)
741)
742)
743)
744)
745)
746)
747)
748)
749)
750)
751)
752)
753)
754)
755)
756)
757)
758)
759)
760)
761)
762)
763)
764)
765)
766)
767)
768)
769)
770)
771)
772)
773)
774)
775)
776)
777)
778)
779)
780)
781)
782)
783)
784)
785)
786)
787)
788)
789)
790)
791)
792)
793)
794)
795)
796)
797)
798)
799)
800)
801)
802)
803)
804)
805)
806)
807)
808)
809)
810)
811)
812)
813)
814)
815)
816)
817)
818)
819)
820)
821)
822)
823)
824)
825)
826)
827)
828)
829)
830)
831)
832)
833)
834)
835)
836)
837)
838)
839)
840)
841)
842)
843)
844)
845)
846)
847)
848)
849)
850)
851)
852)
853)
854)
855)
856)
857)
858)
859)
860)
861)
862)
863)
864)
865)
866)
867)
868)
869)
870)
871)
872)
873)
874)
875)
876)
877)
878)
879)
880)
881)
882)
883)
884)
885)
886)
887)
888)
889)
890)
891)
892)
893)
894)
895)
896)
897)
898)
899)
900)
901)
902)
903)
904)
905)
906)
907)
908)
909)
910)
911)
912)
913)
914)
915)
916)
917)
918)
919)
920)
921)
922)
923)
924)
925)
926)
927)
928)
929)
930)
931)
932)
933)
934)
935)
936)
937)
938)
939)
940)
941)
942)
943)
944)
945)
946)
947)
948)
949)
950)
951)
952)
953)
954)
955)
956)
957)
958)
959)
960)
961)
962)
963)
964)
965)
966)
967)
968)
969)
970)
971)
972)
973)
974)
975)
976)
977)
978)
979)
980)
981)
982)
983)
984)
985)
986)
987)
988)
989)
990)
991)
992)
993)
994)
995)
996)
997)
998)
999)
1000)
```

This calculates the minimum price, maximum price, and average price across all transactions. These statistics provide a quick overview of sales performance.

Q14. What is the risk of using inferSchema=True when source data contains inconsistent date formats?

Ans: Spark may incorrectly identify data types when the input data is inconsistent.

If some records use formats like 2025-01-15 and others use 15/01/2025, Spark may struggle to infer the correct data type.

This can result in:

- Dates being stored as strings
- Parsing errors

- Unexpected null values
- Incorrect query results

For production environments, it is usually safer to define the schema manually and then perform date conversions during data cleaning.

Q15. Write a final processing pipeline that removes duplicates, fills null prices with 0, and calculates total revenue per store.

Ans: A complete pipeline combines cleaning and aggregation steps into a single workflow.

```
from pyspark.sql.functions import sum

final_result = (
    df
    .dropDuplicates()
    .na.fill({"price": 0})
    .groupBy("store_id")
    .agg(sum("price").alias("total_revenue"))
)

final_result.show()
```

```
***
+-----+-----+
|store_id|total_revenue|
+-----+-----+
|  S105  |         3000 |
|  S102  |        57500 |
|  S106  |        40000 |
|  S104  |            0 |
|  S101  |        45800 |
|  S103  |        65000 |
+-----+-----+
```

This pipeline first removes duplicate records, then handles missing prices, and finally calculates the total revenue generated by each store. It demonstrates a typical Spark data processing workflow where cleaning and aggregation are performed together before analysis.